

CyberRAG

AI-Powered Security Intelligence Platform

Technical Document

Multimodal Retrieval-Augmented Generation platform for the cybersecurity industry

Field	Detail
Author	Alina Akhtar
Industry	Cybersecurity
Backend	FastAPI + Postgresql+ Qdrant
Frontend	Streamlit
LLM	Groq — llama-3.3-70b-versatile
Embeddings	sentence-transformers all-MiniLM-L6-v2
Date	31 July 2026

Table of Contents

1. Project Overview and Industry Use Case	3
2. Architecture	3
2.1 Client layer — Streamlit	4
2.2 API layer — FastAPI	4
2.3 Ingestion pipeline	4
2.4 RAG query pipeline.....	5
2.5 Persistence layer	5
3. Authentication System	5
3.1 Registration.....	5
3.2 Login and token creation	6
3.3 get_current_user	6
4. Database Models.....	7
5. Ingestion Pipeline.....	9
5.1 Extractors, one per file type	10
6. Embeddings and Qdrant	11
6.1 Chunking	11
6.2 Embedding model	12
6.3 Qdrant collection and metadata schema	12
7. RAG Chat Endpoint	13
7.1 Retrieve.....	14
7.2 Augment and generate.....	15
7.3 Grounding guarantee	16
8. Conversation History	16
9. Streamlit UI	16
9.1 Session state.....	17
9.2 Attaching the JWT to API calls	17
9.3 Rendering the chat thread	17
10. Challenges and Decisions	17
10.1 file_id showing as None in Qdrant payloads	17
10.2 Zero chunks returned at query time	18
10.3 Keeping the sources field trustworthy	18

1. Project Overview and Industry Use Case

CyberRAG is a multimodal Retrieval-Augmented Generation (RAG) platform built for the cybersecurity industry. It lets an authenticated user create chat rooms (“workspaces”), upload security-related documents and media of any supported type, and ask natural-language questions across everything uploaded to that room. Every answer the system gives is grounded in the uploaded material and is returned together with the exact sources, filename, file type, chunk index, and excerpt, that were used to produce it.

The intended end user is a security analyst, incident responder, or a security-awareness trainer who needs to search across a mixed pile of evidence, threat intelligence reports (PDF), incident write-ups (DOCX), security policies (TXT/Markdown), awareness posters (images), audio/video briefings, and structured indicator lists (CSV), without manually opening every file. Instead of reading through each document, the analyst opens a room, uploads the relevant files, and asks questions such as “what does the policy say about password rotation” or “what threats are listed in the CSV” and receives an answer with a citation trail back to the original file and chunk.

The platform solves two problems at once: it removes the manual effort of searching across many file formats, and it prevents the model from inventing answers if nothing relevant was uploaded, the system explicitly replies “I don't know” rather than guessing.

2. Architecture

The system is organised into five layers: a Streamlit client, a FastAPI REST API guarded by JWT authentication, a multimodal ingestion pipeline, a RAG query pipeline, and a persistence layer split between a relational database, a Qdrant vector store, and local file storage. The approved architecture diagram is reproduced below.

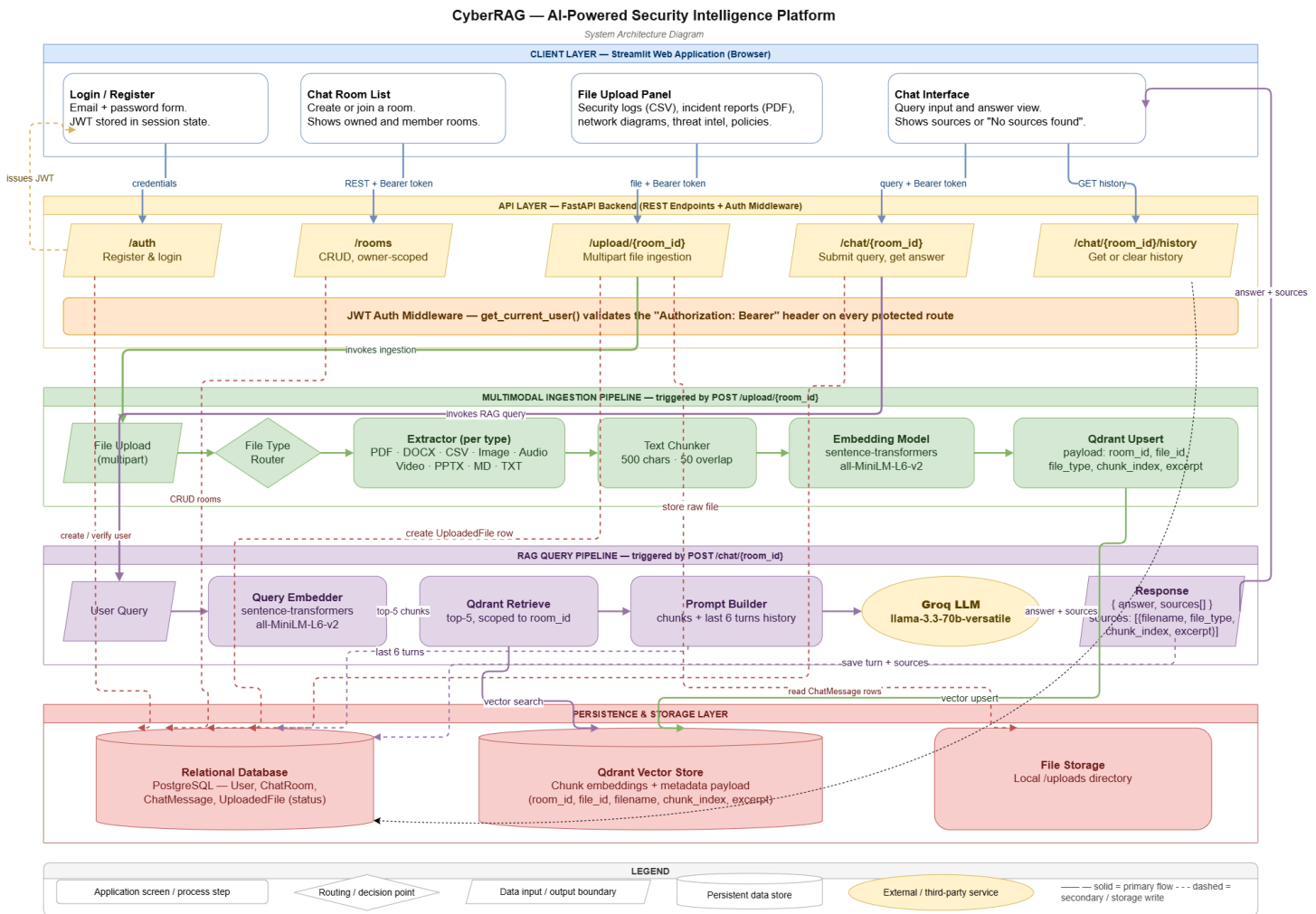


Figure 1. CyberRAG system architecture.

2.1 Client layer — Streamlit

A single-page Streamlit application (`streamlit/streamlit.py`) drives four screens: login/register, the chat room list, the room view (file panel + chat), and a right-hand inspector. It never talks to the database or Qdrant directly — every action is a REST call to the FastAPI backend, made through the thin wrapper functions in `streamlit/api.py`.

2.2 API layer — FastAPI

The backend (`app/main.py`) is a FastAPI application that registers four routers, each owning one group of endpoints:

- **/auth** — register and login (`app/routers/auth.py`)
- **/rooms** — chat room CRUD, owner-scoped (`app/routers/rooms.py`)
- **/upload/{room_id}** — multipart file ingestion (`app/routers/upload.py`)
- **/chat/{room_id}** — ask a question, read/clear history (`app/routers/chat.py`)

Every route except register and login is protected by a `get_current_user` FastAPI dependency that validates the bearer JWT on every call, described in section 3.

2.3 Ingestion pipeline

Triggered by POST /upload/{room_id}: the file is saved to disk, routed to a per-type extractor that returns plain text, split into overlapping chunks, embedded with a sentence-transformers model, and upserted into Qdrant with room, file, and chunk metadata. Full detail in section 5.

2.4 RAG query pipeline

Triggered by POST /chat/{room_id}: the query is embedded, the top-5 matching chunks scoped to the room are retrieved from Qdrant, a prompt is built from those chunks plus the last six turns of conversation history, and the prompt is sent to a Groq-hosted LLM. Full detail in section 7.

2.5 Persistence layer

- **Relational database** — SQLAlchemy models for User, ChatRoom, ChatMessage, and UploadedFile (section 4).
- **Qdrant** — a local, embedded vector store (qdrant_storage/) holding one collection, cyber_rag, of 384-dimension chunk embeddings with metadata payloads.
- **File storage** — uploaded originals are written to a local uploads/ directory with a UUID-prefixed filename; the path is recorded on the UploadedFile row.

3. Authentication System

Authentication is JWT-based. Passwords are hashed with bcrypt (via passlib) and never stored or compared in plain text. All logic lives in app/services/auth.py.

3.1 Registration

POST /auth/register accepts a username, email, and password. The email and password are validated by Pydantic field validators — the password must be at least 8 characters and contain an uppercase letter, a lowercase letter, a digit, and a special character. The handler rejects the request with 400 if the email or username is already taken, otherwise it hashes the password and stores a new User row.

```
@router.post("/register", status_code=201)
def register(user: UserRegister, db: Session = Depends(get_db)):
    existing = db.query(User).filter(User.email == user.email).first()
    if existing:
        raise HTTPException(400, detail="Email already registered")
    existing_username = db.query(User).filter(User.username == user.username).first()
    if existing_username:
        raise HTTPException(status_code=400, detail="Username already taken")
    new_user = User(username=user.username, email=user.email,
                    hashed_password=hash_password(user.password))
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return {"message": "User registered successfully"}
```

```

@router.post("/register", status_code=201)
def register(user:UserRegister,db:Session=Depends(get_db)):
    existing=(db.query(User).filter((User.email==user.email)).first())
    if existing:
        raise HTTPException(
            status_code=400,
            detail="Email already registered"
        )
    existing_username = db.query(User).filter(User.username == user.username).first()

    if existing_username:
        raise HTTPException(
            status_code=400,
            detail="Username already taken"
        )
    new_user=User(username=user.username,email=user.email,hashed_password=hash_password(user.password))
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return {"message":"User registered successfully"}

```

3.2 Login and token creation

POST /auth/login accepts OAuth2 form data (username field carries the email). If the credentials verify, create_access_token encodes the user's email as the sub claim, adds an expiry (ACCESS_TOKEN_EXPIRE_MINUTES from the environment), and signs the token with SECRET_KEY using the HS256-family algorithm configured in ALGORITHM.

```

def create_access_token(data: dict, expires_delta: timedelta = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

```

The returned {access_token, token_type} pair is stored by the Streamlit client in st.session_state.token and attached as an Authorization: Bearer header on every subsequent API call.

```

52 def login(form_data: OAuth2Form):
53     existing_user = db.query(User).filter(User.email == form_data.email).first()
54     if existing_user is None:
55         raise HTTPException(
56             status_code=401,
57             detail="Invalid email or password"
58         )
59     if not verify_password(form_data.password, existing_user.hashed_password):
60         raise HTTPException(
61             status_code=401,
62             detail="Invalid email or password"
63         )
64     token = create_access_token({"sub": existing_user.email})
65     return {
66         "access_token": token,
67         "token_type": "bearer"
68     }

```

3.3 get_current_user

Every protected route depends on `get_current_user`, which decodes the bearer token, extracts the sub (email) claim, and loads the matching User row. Any decode failure, missing claim, or unknown user raises a 401 with a WWW-Authenticate: Bearer header.

```
def get_current_user(token: str = Depends(auth_scheme), db: Session = Depends(get_db)):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid Token",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        email = payload.get("sub")
        if email is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception
    user = db.query(User).filter(User.email == email).first()
    if user is None:
        raise credentials_exception
    return user
```

Because every room, upload, and chat route also filters by `owner_id == current_user.id`, the JWT is what scopes every query to the calling user — a user can never see or query another user's rooms, files, or messages.

```
def get_current_user(token: str = Depends(auth_scheme), db: Session = Depends(get_db)):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid Token",
        headers={"WWW-Authenticate": "Bearer"},
    )

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        email = payload.get("sub")
        if email is None:
            raise credentials_exception

    except JWTError:
        raise credentials_exception

    user = (db.query(User).filter(User.email == email).first())

    if user is None:
        raise credentials_exception
```

4. Database Models

Persistence uses SQLAlchemy models (`app/database/models.py`) against a database configured through the `DATABASE_URL` environment variable, via a standard engine / `SessionLocal` / `get_db` dependency setup in `app/database/database.py`.

```
class User(Base):
    __tablename__ = "users"
```

```

id = Column(Integer, primary_key=True, index=True)
username = Column(String(50), unique=True, index=True, nullable=False)
email = Column(String(100), unique=True, index=True, nullable=False)
hashed_password = Column(String(255), nullable=False)
created_at = Column(DateTime(timezone=True), default=datetime.utcnow)
rooms = relationship("ChatRoom", back_populates="owner")
messages = relationship("ChatMessage", back_populates="user")

```

```

class ChatRoom(Base):
    __tablename__ = "chat_rooms"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(128), unique=True, index=True, nullable=False)
    description = Column(Text, nullable=True)
    created_at = Column(DateTime(timezone=True), default=datetime.utcnow)
    owner_id = Column(Integer, ForeignKey("users.id"))
    owner = relationship("User", back_populates="rooms")
    messages = relationship("ChatMessage", back_populates="room",
                           cascade="all, delete-orphan")
    uploaded_files = relationship("UploadedFile", back_populates="room",
                                 cascade="all, delete-orphan")

```

```

class ChatMessage(Base):
    __tablename__ = "chat_messages"
    id = Column(Integer, primary_key=True, index=True)
    room_id = Column(Integer, ForeignKey("chat_rooms.id"))
    user_id = Column(Integer, ForeignKey("users.id"))
    role = Column(Enum(MessageRole), nullable=False)
    content = Column(Text, nullable=False)
    sources = Column(JSON, nullable=True)
    created_at = Column(DateTime(timezone=True), default=datetime.utcnow)
    room = relationship("ChatRoom", back_populates="messages")
    user = relationship("User", back_populates="messages")

```

```

class UploadedFile(Base):
    __tablename__ = "uploaded_files"
    id = Column(Integer, primary_key=True, index=True)
    room_id = Column(Integer, ForeignKey("chat_rooms.id"))
    filename = Column(String(255), nullable=False)
    file_type = Column(String(50), nullable=False)
    file_path = Column(String(255), nullable=False)
    status = Column(Enum(FileStatus), default=FileStatus.PROCESSING)
    error_message = Column(Text, nullable=True)
    uploaded_at = Column(DateTime(timezone=True), default=datetime.utcnow)
    room = relationship("ChatRoom", back_populates="uploaded_files")

```

Relationships: a User owns many ChatRoom rows and authors many ChatMessage rows. A ChatRoom owns many ChatMessage and UploadedFile rows, both deleted automatically (cascade="all, delete-orphan") when the room is deleted. UploadedFile.status is an enum (PROCESSING, UPLOADED, FAILED) that lets the Streamlit sidebar show a live status badge per file, and ChatMessage.sources is a JSON column so each assistant turn carries its own citation list alongside the message text.


```

class User(Base):
    __tablename__ = "users"
    id=Column(Integer,primary_key=True,index=True)
    username=Column(String(50),unique=True,index=True,nullable=False)
    email=Column(String(100),unique=True,index=True,nullable=False)
    hashed_password=Column(String(255),nullable=False)
    created_at=Column(DateTime(timezone=True),default=datetime.utcnow)
    rooms=relationship("ChatRoom",back_populates="owner")
    messages=relationship("ChatMessage",back_populates="user")

class ChatRoom(Base):
    __tablename__ = "chat_rooms"
    id=Column(Integer,primary_key=True,index=True)
    name=Column(String(128),index=True,nullable=False)
    description=Column(Text,nullable=True)
    created_at=Column(DateTime(timezone=True),default=datetime.utcnow)
    owner_id=Column(Integer,ForeignKey("users.id"))
    owner=relationship("User",back_populates="rooms")
    messages=relationship("ChatMessage",back_populates="room",cascade="all, delete-orphan")
    uploaded_files=relationship("UploadedFile",back_populates="room",cascade="all, delete-orphan")

class ChatMessage(Base):
    __tablename__ = "chat_messages"
    id=Column(Integer,primary_key=True,index=True)
    room_id=Column(Integer,ForeignKey("chat_rooms.id"))
    user_id=Column(Integer,ForeignKey("users.id"))
    role=Column(Enum(MessageRole),nullable=False)
    content=Column(Text,nullable=False)
    sources=Column(JSON,nullable=True)
    created_at=Column(DateTime(timezone=True),default=datetime.utcnow)
    room=relationship("ChatRoom",back_populates="messages")
    user=relationship("User",back_populates="messages")

class UploadedFile(Base):
    __tablename__ = "uploaded_files"
    id=Column(Integer,primary_key=True,index=True)

```

5. Ingestion Pipeline

POST /upload/{room_id} (app/routers/upload.py) is the single entry point for every supported file type. It verifies the room belongs to the caller, checks the file extension against an allow-list, saves the file under a UUID-prefixed name in uploads/, creates an UploadedFile row with status PROCESSING, and then dispatches to the matching extractor by extension:

```

if extension == "csv":    texts = extract_csv(save_path)
elif extension == "docx": texts = extract_docx(save_path)
elif extension == "pdf":  texts = extract_pdf(save_path)
elif extension == "txt":  texts = extract_txt(save_path)
elif extension == "md":   texts = extract_md(save_path)
elif extension == "pptx": texts = extract_pptx(save_path)
elif extension in ["jpg", "jpeg", "png"]: texts = extract_image(save_path)
elif extension in ["mp3", "wav", "flac"]: texts = extract_audio(save_path)
elif extension in ["mp4", "avi", "mov"]:  texts = extract_video(save_path)

```

Every extractor returns a list of plain-text strings (one per logical unit — a whole document, or one entry per slide/page). Each string is then split into 500-character chunks with a 50-character overlap by `chunk_text()`, embedded, and upserted to Qdrant. If any step throws, the exception is caught, the DB transaction is rolled back, the partially-written file is deleted from disk, and the row is left as PROCESSING while a 500 is returned to the caller.

```

def upload_file(
    )

    db.add(uploaded)
    db.commit()
    db.refresh(uploaded)

    if extension == "csv":
        texts = extract_csv(save_path)

    elif extension == "docx":
        texts = extract_docx(save_path)

    elif extension == "pdf":
        texts = extract_pdf(save_path)

    elif extension == "txt":
        texts = extract_txt(save_path)

    elif extension == "md":
        texts = extract_md(save_path)

    elif extension == "pptx":
        texts = extract_pptx(save_path)

    elif extension in ["jpg", "jpeg", "png"]:
        texts = extract_image(save_path)

    elif extension in ["mp3", "wav", "flac"]:
        texts = extract_audio(save_path)

    elif extension in ["mp4", "avi", "mov"]:
        texts = extract_video(save_path)

    chunks = []

```

5.1 Extractors, one per file type

- **PDF** (`pymupdf4llm/PyMuPDF`, `app/services/ingestion/pdf.py`) — opens the document with `fitz` and concatenates `get_text()` across every page into a single string.

```

def extract_pdf(file_path: str):
    doc = fitz.open(file_path)
    text = ""
    for page in doc:
        text += page.get_text()
    doc.close()
    return [text]

```

- **Word / DOCX** (`python-docx`, `app/services/ingestion/docx.py`) — reads every non-empty paragraph, then walks every table and joins each row's non-empty cells with `" | "` so tabular data (prices, specs, timelines) is not silently dropped.

```

def extract_docx(file_path: str):
    doc = Document(file_path)
    parts = []
    for paragraph in doc.paragraphs:
        if paragraph.text.strip():
            parts.append(paragraph.text)
    for table in doc.tables:
        for row in table.rows:
            row_text = " | ".join(cell.text.strip() for cell in row.cells if cell.text.strip())
            if row_text:
                parts.append(row_text)
    return ["\n".join(parts)]

```

- **CSV (pandas, app/services/ingestion/csv_file.py)** — loads the file into a DataFrame and serialises it with `df.to_string()` so the LLM sees the data as an aligned text table.
- **PowerPoint (python-pptx, app/services/ingestion/pptx.py)** — returns one text string per slide (all shape text on that slide concatenated), so retrieval and citations can point at a specific slide rather than the whole deck.
- **Markdown / plain text (app/services/ingestion/markdown.py, txt.py)** — read the file as UTF-8 and return the raw text as a single-element list; no markup stripping is performed.
- **Images (Groq vision model, app/services/ingestion/image.py)** — the image is base64-encoded and sent to a Groq vision-capable chat model with an instruction to describe the image and transcribe any visible text or security-relevant data; the model's description becomes the searchable text for that file.

```
response = client.chat.completions.create(
    model="qwen/qwen3.6-27b",
    messages=[{"role": "user", "content": [
        {"type": "image_url", "image_url": {"url": f"data:{mime};base64,{image_data}"}}],
        {"type": "text", "text": "Describe this image in detail. Extract all visible text, "
                                "data, labels, and any relevant cybersecurity information."}
    ]}]
)
```

- **Audio (Groq Whisper large-v3, app/services/ingestion/audio.py)** — the file is streamed directly to the Whisper transcription endpoint and the returned transcript text is used as-is.
- **Video (moviepy + Groq Whisper, app/services/ingestion/video.py)** — moviepy's `VideoFileClip` extracts only the audio track to a temporary `.wav` file (the full video binary is never sent to any API), that file is transcribed with the same Whisper call as audio, and the temporary file is deleted afterwards.

```
def extract_video(file_path: str):
    audio_path = file_path + "_audio.wav"
    clip = VideoFileClip(file_path)
    clip.audio.write_audiofile(audio_path, logger=None)
    clip.close()
    with open(audio_path, "rb") as f:
        transcription = client.audio.transcriptions.create(
            model="whisper-large-v3", file=f, response_format="text")
    os.remove(audio_path)
    return [transcription]
```

6. Embeddings and Qdrant

6.1 Chunking

`chunk_text()` (`app/services/chunker.py`) performs fixed-size, overlapping character chunking: 500 characters per chunk with a 50-character overlap between consecutive chunks, so a sentence that straddles a chunk boundary is not lost from either side.

```
def chunk_text(text: str, chunk_size: int = 500, overlap: int = 50) -> List[str]:
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks
```

```

from typing import List

def chunk_text(text:str, chunk_size:int=500,overlap: int=50)->List[str]:
    chunks=[]
    start=0
    while start<len(text):
        end=start+chunk_size
        chunks.append(text[start:end])
        start+=chunk_size-overlap
    return chunks

```

6.2 Embedding model

app/services/embedder.py loads a single shared instance of the sentence-transformers all-MiniLM-L6-v2 model (384-dimensional embeddings) at import time and exposes get_embedding(text), used by both the upload path and the query path so vectors from both sides live in the same space.

```

model = SentenceTransformer('all-MiniLM-L6-v2')
def get_embedding(text: str):
    return model.encode(text).tolist()

```

```

app > services > embedder.py > ...
1  from sentence_transformers import SentenceTransformer
2
3  model= SentenceTransformer('all-MiniLM-L6-v2')
4  def get_embedding(text:str):
5      return model.encode(text).tolist()
6

```

6.3 Qdrant collection and metadata schema

app/services/qdrant_db.py opens an embedded (local, file-backed) Qdrant client rooted at qdrant_storage/ and lazily creates a single collection, cyber_rag, with 384-dimensional vectors and cosine distance, if it does not already exist.

```

collection_name = "cyber_rag"
client = QdrantClient(path="qdrant_storage")
existing = [c.name for c in client.get_collections().collections]
if collection_name not in existing:
    client.create_collection(
        collection_name=collection_name,
        vectors_config=VectorParams(size=384, distance=Distance.COSINE)
    )

```

Each chunk is upserted as one PointStruct with a random UUID id, its embedding vector, and a metadata payload of room_id, file_id, filename, chunk_index, file_type, and the chunk's raw text:

```

app > services > qdrant_db.py > ...
1  from qdrant_client import QdrantClient
2  from qdrant_client.models import Distance, VectorParams
3
4  collection_name="cyber_rag"
5  client=QdrantClient(path="qdrant_storage")
6  existing = [c.name for c in client.get_collections().collections]
7  if collection_name not in existing:
8      client.create_collection(
9          collection_name=collection_name,
10         vectors_config=VectorParams(size=384, distance=Distance.COSINE)
11     )
12
13
14

```

```

client.upsert(
    collection_name=collection_name,
    points=[PointStruct(
        id=uuid.uuid4(),
        vector=vector,
        payload={
            "room_id": room_id, "file_id": uploaded.id,
            "filename": uploaded.filename, "chunk_index": i,
            "file_type": extension, "text": chunk
        }
    )]
)

```

The room_id in the payload is what scopes retrieval to a single chat room, and file_id is what lets a file (and only that file's vectors) be deleted independently both re-uploading a file with the same name and deleting a file first remove the old vectors with a Qdrant Filter on file_id before touching the relational rows.

7. RAG Chat Endpoint

POST /chat/{room_id} (app/routers/chat.py) implements the full retrieve → augment → generate loop and is the only place an answer is produced.

```

@router.post("/{room_id}")
def chat(room_id: int, chat_request: ChatRequest, db: Session = Depends(get_db),
        current_user: User = Depends(get_current_user)):
    room = db.query(ChatRoom).filter(ChatRoom.id == room_id,
                                     ChatRoom.owner_id == current_user.id).first()
    if not room:
        raise HTTPException(status_code=404, detail="Room not found")

    history = (db.query(ChatMessage).filter(ChatMessage.room_id == room_id)
              .order_by(ChatMessage.created_at.desc()).limit(6).all())
    history.reverse()

    chunks = search_documents(room_id, chat_request.query)

    if not chunks:
        answer, sources = "I don't know.", []
    else:
        answer = generate_answer(chat_request.query, chunks, history)
        if answer.strip().lower() == "i don't know.":
            sources = []

```

```

else:
    sources = [{
        "filename": c["filename"], "file_type": c["file_type"],
        "chunk_index": c["chunk_index"], "excerpt": c["text"][:150]
    } for c in chunks]

db.add(ChatMessage(room_id=room_id, role="user", content=chat_request.query))
db.add(ChatMessage(room_id=room_id, role="assistant", content=answer, sources=sources))
db.commit()

return {"answer": answer, "sources": sources}

```

```

> routers > chat.py > delete_history
2 def chat(
1     else:
2
3         sources = []
4
5         for chunk in chunks:
6
7             sources.append({
8                 "filename": chunk["filename"],
9                 "file_type": chunk["file_type"],
10                "chunk_index": chunk["chunk_index"],
11                "excerpt": chunk["text"][:150]
12            })
13
14        user_message = ChatMessage(
15            room_id=room_id,
16            role="user",
17            content=chat_request.query
18        )
19
20        assistant_message = ChatMessage(
21            room_id=room_id,
22            role="assistant",
23            content=answer,
24            sources=sources
25        )
26
27        db.add(user_message)
28        db.add(assistant_message)
29
30        db.commit()
31
32        return {
33            "answer": answer,
34            "sources": sources
35        }
36

```

7.1 Retrieve

`search_documents()` (`app/services/retriever.py`) embeds the query with the same sentence-transformers model, then calls Qdrant's `query_points` with a `Filter` on `room_id` and `limit=5`, returning the payload of each of the top-5 matches by cosine similarity. If no chunk exists for the room at all, the endpoint short-circuits straight to “I don't know.” with an empty sources list the LLM is never called.

```

pp > services > retriever.py > search_documents
1  from app.services.qdrant_db import client
2  from app.services.embedder import model
3  from qdrant_client.models import Filter, FieldCondition, MatchValue
4
5  def search_documents(room_id:int,query:str):
6      vector=model.encode(query).tolist()
7
8      results=client.query_points(
9          collection_name="cyber_rag",
10         query=vector,
11         limit=5,
12         query_filter=Filter(
13             must=[
14                 FieldCondition(
15                     key="room_id",
16                     match=MatchValue(value=room_id)
17                 )
18             ]
19         )
20     )
21
22     return [point.payload for point in results.points]

```

7.2 Augment and generate

generate_answer() (app/services/rag.py) joins the retrieved chunks' text into a single context block, opens the message list with a system prompt that restricts the model to the given context and forces the exact phrase "I don't know." when the context does not answer the question, appends the last six turns of history, and appends the current question with its context as the final user turn.

```

messages = [{"role": "system", "content": (
    "You are a cybersecurity intelligence assistant. "
    "Answer ONLY using the provided context. "
    "If the answer is not in the context, reply exactly: I don't know. "
    "Do not use outside knowledge. Do not make up information."
)}]
for chat in history:
    messages.append({"role": chat.role, "content": chat.content})
messages.append({"role": "user", "content": f"Context:\n{context}\n\nQuestion: {question}"})
response = client.chat.completions.create(model="llama-3.3-70b-versatile", messages=messages)
return response.choices[0].message.content

```

sources is built directly from the same five retrieved chunks that were sent to the model not from anything the model claims so the citation trail is guaranteed to reflect what was actually retrieved. sources is always present in the response, as either a populated list or an explicit empty list, never null or omitted.

```

3 from groq import Groq
4 load_dotenv()
5
6 client = Groq(
7     api_key=os.getenv("GROQ_API_KEY")
8 )
9
10 def generate_answer(question, chunks, history):
11     context = "\n\n".join([chunk["text"] for chunk in chunks])
12
13     messages = [
14         {
15             "role": "system",
16             "content": (
17                 "You are a cybersecurity intelligence assistant. "
18                 "Answer ONLY using the provided context. "
19                 "If the answer is not in the context, reply exactly: I don't know. "
20                 "Do not use outside knowledge. Do not make up information."
21             )
22         }
23     ]
24     for chat in history:
25         messages.append({
26             "role": chat.role if hasattr(chat, "role") else chat["role"],
27             "content": chat.content if hasattr(chat, "content") else chat["content"]
28         })
29     messages.append({
30         "role": "user",
31         "content": f"Context:\n{context}\n\nQuestion: {question}"
32     })
33
34     response = client.chat.completions.create(
35         model="llama-3.3-70b-versatile",
36         messages=messages
37     )
38     return response.choices[0].message.content

```

7.3 Grounding guarantee

- No chunks retrieved for the room → answer is “I don't know.” and sources is [].
- Chunks retrieved but the model still cannot answer from them → model returns “I don't know.” and sources is forced to [] even though chunks existed.
- Chunks retrieved and the model answers → sources lists exactly the chunks that were sent to the model, each with filename, file_type, chunk_index, and a 150-character excerpt.

8. Conversation History

Every chat turn — both the user's message and the assistant's answer, including its sources JSON is persisted as a ChatMessage row. Inside the chat handler, the six most recent rows for the room are fetched ordered newest-first, then reversed back into chronological order before being spliced into the Groq message list as prior turns, giving the model short-term memory of the conversation without resending the entire history on every call.

GET /chat/{room_id}/history returns the full, paginated (skip/limit) history for a room ordered oldest-first, including each message's stored sources (or [] if none), for the Streamlit client to render when a room is opened. DELETE /chat/{room_id}/history is owner-only: it deletes every ChatMessage row for the room, deletes every associated UploadedFile's vectors from Qdrant and its file from disk, then deletes the UploadedFile rows effectively resetting the room to empty.

9. Streamlit UI

9.1 Session state

Streamlit re-executes the whole script on every interaction, so all state that must survive a rerun is kept in `st.session_state`. A defaults dictionary seeds `token`, `user`, `rooms`, `selected_room`, `messages`, `files`, `page`, `auth_mode`, and `upload_key` on first load:

```
defaults = {
    "token": None, "user": None, "rooms": [], "selected_room": None,
    "messages": [], "files": [], "page": "login", "auth_mode": "login",
    "upload_key": 0
}
for key, value in defaults.items():
    if key not in st.session_state:
        st.session_state[key] = value
```

`st.session_state.page` ("login", "rooms", or "room") drives which screen function is called at the bottom of the script. `logout()` clears every key and sends the user back to "login".

9.2 Attaching the JWT to API calls

On a successful login response, the access token is written into `st.session_state.token`. Every subsequent call from `api.py` accepts that token and attaches it as a standard Authorization: Bearer header via a shared helper:

```
def auth_headers(token):
    return {"Authorization": f"Bearer {token}"}

def get_rooms(token):
    return requests.get(f"{API_URL}/rooms", headers=auth_headers(token))
```

Every screen that needs backend data (rooms, files, history, chat) passes `st.session_state.token` into the corresponding `api.py` function, so the token never needs to be looked up or re-entered per request.

9.3 Rendering the chat thread

The room view splits into a left file panel, a centre chat panel, and a right inspector panel. The chat panel iterates `st.session_state.messages` and calls `render_message()` for each one: user turns render as a plain chat bubble; assistant turns render the answer text followed by a collapsible “Sources · N” expander built by `render_sources()`.

```
sources = message.get("sources", [])
with st.expander(f"Sources · {len(sources)}", expanded=False):
    render_sources(sources)
```

`render_sources()` never hides the Sources section: if the list is empty it explicitly renders “No sources found” instead of omitting the expander, matching the same guarantee enforced on the backend.

New chat input from `st.chat_input()` is appended to session state immediately as a user turn for instant feedback, then `send_chat()` posts it to `POST /chat/{room_id}`; on a 200/201 response the assistant turn (with its sources) is appended and the page reruns to redraw the thread. On room entry, `load_room()` calls `get_history()` and `get_files()` to hydrate `st.session_state.messages` and `st.session_state.files` from the backend so history and file status badges survive navigation and page reloads.

10. Challenges and Decisions

10.1 file_id showing as None in Qdrant payloads

Early in development, chunks upserted to Qdrant during ingestion sometimes carried `file_id`: `None` in their payload. The cause was ordering: the upsert loop was writing `uploaded.id` into the payload before the `UploadedFile` row had actually been committed and refreshed from the database, so the ORM object did not yet have its auto-generated primary key populated — it was still `None` at the point the vectors were built. Because `file_id` is what later lets a single file's vectors be found and deleted (on delete or on re-upload of a same-named file), any chunk written with a `None` `file_id` became orphaned and undeletable through the normal flow. The fix was to move `db.commit()` and `db.refresh(uploaded)` immediately after the `UploadedFile` row is created and before any extraction or embedding work begins, guaranteeing `uploaded.id` is a real integer by the time it is written into every chunk's payload.

10.2 Zero chunks returned at query time

A related issue showed up as `search_documents()` returning zero chunks for a room that clearly had files uploaded, which made the chat endpoint fall back to “I don't know.” even though relevant content existed. This traced back to three separate causes rather than one bug: some files silently failed inside their extractor (an exception during extraction rolled back the DB row and removed the file, so no chunks were ever created for it in the first place); a few chunks were written under a mismatched filename or `file_type` compared to what the `UploadedFile` row and later lookups expected, so they existed in Qdrant but did not line up with the file the user thought they had uploaded; and the image ingestion path occasionally failed outright (a vision-model call error or an unsupported image variant), which meant `extract_image()` produced no text, no chunks, and nothing for Qdrant to return for that file. Each case was fixed at its source — tightening the exception handling and cleanup in the upload route, keeping filename/`file_type` consistent between the `UploadedFile` row and the payload written at upsert time, and hardening the image extractor — rather than papering over the symptom in the retriever.

10.3 Keeping the sources field trustworthy

The hardest problem was guaranteeing that the `sources` field is trustworthy rather than something the LLM could contradict or omit an early version derived sources from parsing the model's own answer text, which broke whenever the model rephrased or summarised a filename. The fix was to decouple sources entirely from the model's output: `sources` is now built directly from the same chunk list that was retrieved from Qdrant and handed to the model as context, and it is only zeroed out in the two well-defined cases where nothing was actually used to answer (no chunks retrieved, or the model itself returned the exact “I don't know.” sentinel). This also made the “no sources → no null/omitted field” requirement trivial to satisfy, since the code always constructs a list populated or empty rather than an optional value.